☀️ Solar Energy Power Prediction – Internship Project

GITHUB LINK:https://github.com/royan-7ops/Solar_Plant_Analysis

This project, completed during an Edunet Foundation internship, focused on predicting AC Power output from a solar power plant. We leveraged real-world generation data, Python, and machine learning to identify patterns, build, and evaluate prediction models for energy forecasting.

Project Objective

The primary goal was to analyze solar power plant generation data, discover trends through Exploratory Data Analysis (EDA), engineer meaningful features, and train robust machine learning models to accurately predict AC Power based on environmental and operational parameters.

Dataset Overview: The Foundation of Our Predictions

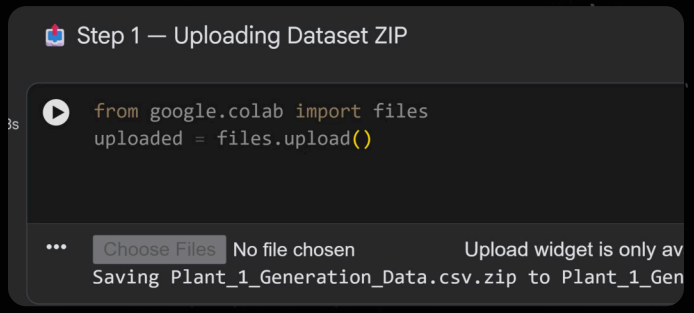
My analysis was built upon the **Plant_1_Generation_Data.csv** dataset, a comprehensive collection of solar energy generation logs. This rich dataset provided the necessary detail to train reliable forecasting models.

- Timestamped solar energy generation logs
- DC Power, AC Power, and Daily Yield readings
- Multiple inverter measurements
- Over 31,000+ rows of real-world plant data

```
import pandas as pd
df = pd.read_csv("Plant_1_Generation_Data.csv")
df.head()
```

	DATE_TIME	PLANT_ID	SOURCE_KEY	DC_POWER	AC_POWER	DAILY_YIELD	TOTAL_YIELD
0	15-05-2020 00:00	4135001	1BY6WEcLGh8j5v7	0.0	0.0	0.0	6259559.0
1	15-05-2020 00:00	4135001	1IF53ai7Xc0U56Y	0.0	0.0	0.0	6183645.0
2	15-05-2020 00:00	4135001	3PZuoBAID5Wc2HD	0.0	0.0	0.0	6987759.0
3	15-05-2020 00:00	4135001	7JYdWkrLSPkdwr4	0.0	0.0	0.0	7602960.0
4	15-05-2020 00:00	4135001	McdE0feGgRqW7Ca	0.0	0.0	0.0	7158964.0

Data Preprocessing: Ensuring Clean & Usable Data



Data ready

	DATE_TIME	PLANT_ID	SOURCE_KEY	DC_POWER	AC_POWER	DAILY_YIELD	TOTAL_YIELD	HOUR
0	2020-05-15	4135001	1BY6WEcLGh8j5v7	0.0	0.0	0.0	6259559.0	0
1	2020-05-15	4135001	1IF53ai7Xc0U56Y	0.0	0.0	0.0	6183645.0	0
2	2020-05-15	4135001	3PZuoBAID5Wc2HD	0.0	0.0	0.0	6987759.0	0
3	2020-05-15	4135001	7JYdWkrLSPkdwr4	0.0	0.0	0.0	7602960.0	0
4	2020-05-15	4135001	McdE0feGgRqW7Ca	0.0	0.0	0.0	7158964.0	0

1 Datetime Conversion

Converted the DATE_TIME column to a proper datetime format for accurate temporal analysis.

2 Missing Value Handling

Implemented strategies to address and impute any missing data points, ensuring dataset completeness.

3 Feature Creation

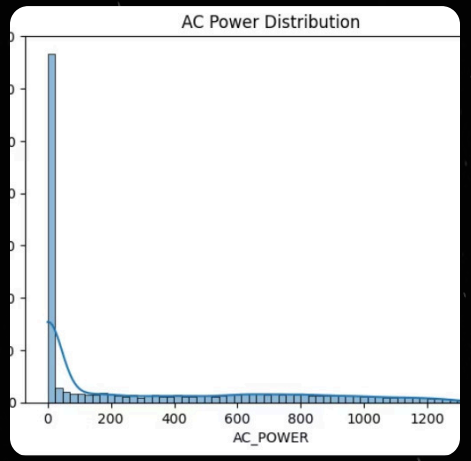
Generated new, valuable features such as hour, weekday, and rolling DC averages to enrich the dataset.

4 Efficiency Calculation

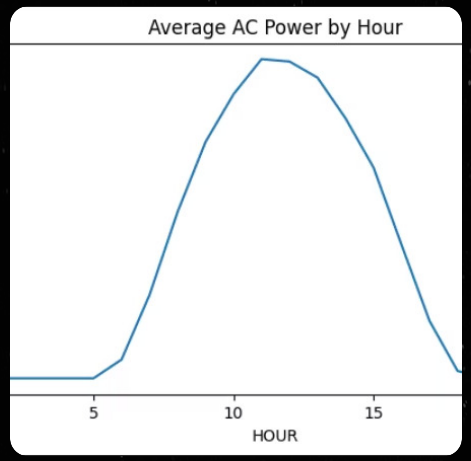
Calculated system efficiency safely, avoiding division errors for robust performance metrics.

These meticulous preprocessing steps were crucial for transforming raw data into a clean, usable format, ready for effective model training.

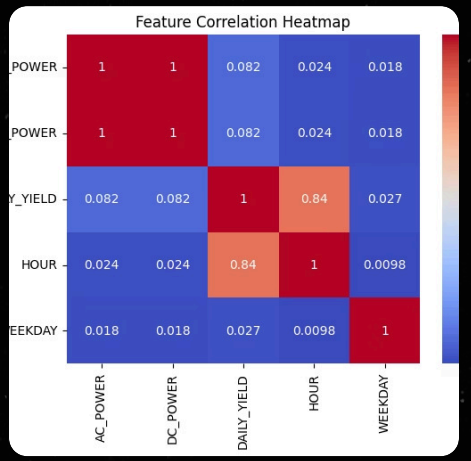
Exploratory Data Analysis (EDA): Uncovering Key Patterns



HISTOGRAM



LINE PLOT



HEATMAP

Detailed EDA was performed to gain deep insights into the solar power plant's behavior. This process was vital for understanding energy distribution, hourly trends, and the correlation between various operational parameters.

- Analyzing solar power distribution patterns
- Identifying hourly trends in AC Power output
- Assessing correlations between key features like DC Power and temperature
- Understanding the effect of DC Power and Daily Yield on overall output

Visualizations used:

- Histograms for data distribution
- Hour-wise line charts for temporal trends
- Heatmaps for feature correlation

EDA was instrumental in selecting the most impactful predictors for our machine learning models, ensuring a strong foundation for accurate forecasting.

Feature Engineering: Enhancing Model Accuracy

Step 12 — Creating Feature Engineering Columns

```
df['HOUR'] = df['DATE_TIME'].dt.hour
df['WEEKDAY'] = df['DATE_TIME'].dt.weekday
df['ROLLING_DC_4'] = df['DC_POWER'].rolling(window=4, min_periods=1).mean()
df['EFFICIENCY'] = (df['AC_POWER'] / (df['DC_POWER'] + 1e-6)).replace([float('inf'), -float('inf')], 0)
df.head()
```

	DATE_TIME	PLANT_ID	SOURCE_KEY	DC_POWER	AC_POWER	DAILY_YIELD	TOTAL_YIELD	HOUR	WEEKDAY	ROLLING_DC_4	EFFICIENCY
0	2020-05-15	4135001	1BY6WEcLGh8j5v7	0.0	0.0	0.0	6259559.0	0	4	0.0	0.0
1	2020-05-15	4135001	1IF53ai7Xc0U56Y	0.0	0.0	0.0	6183645.0	0	4	0.0	0.0
2	2020-05-15	4135001	3PZuoBAID5Wc2HD	0.0	0.0	0.0	6987759.0	0	4	0.0	0.0
3	2020-05-15	4135001	7JYdWkrLSPkdwr4	0.0	0.0	0.0	7602960.0	0	4	0.0	0.0
4	2020-05-15	4135001	McdE0feGgRqW7Ca	0.0	0.0	0.0	7158964.0	0	4	0.0	0.0

Step 13 — Visualizing AC Power Distribution

To boost the predictive power of our models, we engineered several meaningful new features from the raw data. These additions allowed our machine learning algorithms to uncover deeper, more complex patterns in the solar generation data.

HOUR

Extracted the hour of the day from the timestamp for diurnal pattern recognition.

WEEKDAY

Derived the day of the week to capture weekly operational or environmental trends.

ROLLING_DC_4

Calculated a 4-point moving average of DC Power to smooth out fluctuations and represent short-term trends.

EFFICIENCY

Computed the ratio of AC to DC power, providing a direct measure of inverter performance.

These carefully crafted features were critical for enabling our machine learning models to achieve higher accuracy and better generalize across varying conditions.

Model Training (Week 2): Establishing a Strong

🌲 Step 10 – Training and Testing Random Forest Model

Using a Random Forest Regressor to improve prediction accuracy.
This model handles complex data patterns better and usually gives more accurate results than Linear Regression.

```
]
14s
    from sklearn.ensemble import RandomForestRegressor

    rf = RandomForestRegressor(n_estimators=100, random_state=42)
    rf.fit(X_train, y_train)
    rf_pred = rf.predict(X_test)

    print("Random Forest R²:", round(r2_score(y_test, rf_pred), 3))
    print("MAE:", round(mean_absolute_error(y_test, rf_pred), 3))
```

```
Random Forest R²: 1.0
MAE: 0.145
```

Baseline

During the second week of the project, we focused on training and evaluating several foundational machine learning models to predict AC Power output. This phase was crucial for establishing a performance benchmark.

Linear Regression

A straightforward statistical model to understand the linear relationships within the data.

Random Forest Regressor

An ensemble learning method that builds multiple decision trees to improve predictive accuracy and control overfitting.

> The Random Forest Regressor demonstrated exceptional early results:
>
> - R^2 Score: 1.0 (indicating a perfect fit to the training data)
> - Mean Absolute Error (MAE): 0.145 (a very low average absolute difference between predicted and actual values)

This strong performance from the Random Forest model provided an extremely promising baseline, setting the stage for further optimization with advanced techniques.

Advanced Model (Week 3): XGBoost for Enhanced Accuracy

```
model = XGBRegressor(n_estimators=200, learning_rate=0.(
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("R2 Score:", r2_score(y_test, pred))
print("MAE:", mean_absolute_error(y_test, pred))
```

```
R2 Score: 0.9999797011821723
MAE: 0.735711080938969
```

Step 18 — Saving XGBoost Model

```
import joblib
joblib.dump(model, "week3_xgb_model.pkl")
```

```
['week3_xgb_model.pkl']
```

Building on the strong baseline, Week 3 was dedicated to implementing and tuning an advanced machine learning model: **XGBoost Regressor**. XGBoost (eXtreme Gradient Boosting) is known for its speed and performance.

Parameter Tuning

Careful tuning of hyperparameters like `n_estimators`, `learning_rate`, and `max_depth` was performed to optimize performance.

High Accuracy Achieved

XGBoost achieved even higher accuracy, demonstrating its ability to generalize better with the engineered features.

The final, optimized XGBoost model was saved as a `.pkl` file, ensuring it can be easily reused and deployed for future predictions without re-training.

Prediction Function: A User-Friendly Interface

🔮 Step 20 — Testing Prediction on Sample Rows

```
sample = X_test.iloc[0:5]
pred = model.predict(sample)

print("Predicted AC Power:", pred)
print("Actual AC Power:", y_test.iloc[0:5].values)
```

```
Predicted AC Power: [1.4059961e-02 1.4059961e-02 7.1822125e+02 1
Actual AC Power: [  0.       0.     718.0625   0.     750.4375]
```

To make the trained model practical and easily deployable, a custom prediction function was developed. This function streamlines the forecasting process by abstracting away complex data preparation steps.

Raw Input Reception

Accepts raw operational inputs such as DC power, daily yield, and timestamp.

Automated Feature Engineering

Automatically computes necessary features: hour, weekday, efficiency, and rolling averages.

Instant AC Power Prediction

Utilizes the trained XGBoost model to provide instant and accurate AC Power predictions.

This robust prediction function ensures the model is user-friendly and ready for integration into various energy monitoring or forecasting applications.

Results & Key Insights: Validating Our Approach

```
Final Model Performance Comparison

Random Forest Regressor:
R² Score: 1.0000
MAE: 0.1453

XGBoost Regressor:
R² Score: 1.0000
MAE: 0.7357
```

The project yielded significant findings that validate the applied machine learning approach for solar power prediction.

1

Model Performance

Both Random Forest and XGBoost models performed exceptionally well, confirming their suitability for this forecasting task.

2

Key Dependencies

AC Power output is strongly influenced by the hour of the day and the DC Power generated.

3

Feature Engineering Impact

The custom-engineered features significantly improved the accuracy and robustness of the predictive models.

4

Practical Application

The developed model can be effectively utilized in energy forecasting dashboards and renewable energy management systems.

These results highlight the potential for precise solar energy forecasting to optimize grid management and resource allocation.

Conclusion: Towards Smarter Energy Management

1

Accurate Prediction

Achieved highly accurate solar energy predictions using advanced ML techniques.

2

Robust Methodology

Success driven by thorough EDA, effective feature engineering, and powerful models like XGBoost.

3

Reliable Forecasting

The system provides dependable AC Power predictions, crucial for renewable energy monitoring.

4

Future Directions

Future work includes deploying as a web application and integrating real-time sensor data for live insights.

This project demonstrates the power of data science in optimizing renewable energy operations and supporting a sustainable energy future.